



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Project Report

FPGA BASED SYSTEM DESIGN

BECE406E

Pomodoro Timer: An FPGA and Micro-controller Co-Design

Team Members – Rohan Venkataraman (24BEC0174)

Joseph Justin (24BEC0227)

Sanat Vashisht (24BEC0411)

Varun Shankar (24BEC0230)

1. Introduction

The aim of the project is to design and implement a Pomodoro Timer using the Vicharak Shrike Lite development board, which integrates a Renesas SLG47910V FPGA with 1120 LUTs alongside a Raspberry Pi RP2040 microcontroller.

The Pomodoro Technique is a time management method in which work sessions are divided into focused intervals, typically 25 minutes long, separated by short breaks. Implementing this as an embedded system allowed us to explore how digital logic in RTL can work alongside higher-level microcontroller firmware to produce a complete and functional device.

The core idea behind this project is to use the FPGA for all real-time deterministic operations such as clock division, BCD countdown, UART transmission, buzzer control, and LED sequencing, while the RP2040 handles non-deterministic tasks like display rendering, user feedback, and peripheral management. Communication between the two chips is done over a UART link, where the FPGA periodically sends the current timer value as a formatted string and the RP2040 parses it and updates an SH1106 OLED display.

2. Project Objective

The primary objective of this project is to build a functional Pomodoro Timer on an FPGA-MCU hybrid platform that demonstrates a clean separation between hardware logic and firmware. Specific objectives include:

- Implement a precise BCD countdown timer in Verilog RTL running on the SLG47910V FPGA.
- Design a custom UART transmitter module in Verilog to send timer data from the FPGA to the RP2040 at 9600 baud over a 50 MHz clock.
- Parse the received UART string on the RP2040 and render it on an SH1106 OLED display using the Adafruit GFX library.
- Implement work-to-break state transitions in RTL, including automatic reload of timer values when a session ends.
- Drive a passive buzzer and coin vibration motor from the FPGA to provide auditory and haptic feedback on state transitions.
- Integrate LED indicators (green and red) to reflect the current timer state visually.
- Flash the FPGA bitstream from LittleFS storage on the RP2040 automatically at every boot using the Shrike library.

3. Problem Statement

Most FPGA-based timer implementations rely on either a pure HDL approach with seven-segment displays or a pure microcontroller approach with no hardware logic involvement. Neither of these demonstrates the practical advantage of a heterogeneous system where both a programmable logic device and a microcontroller are used together according to their respective strengths.

The challenge addressed in this project is how to correctly partition a real-time application between an FPGA and a microcontroller. The FPGA is well-suited for precise timing, synchronous state machines, and deterministic output control, but it is not practical for driving an I2C display or managing complex string rendering. The RP2040 on the other hand is capable of running libraries and handling display drivers with ease, but it lacks the real-time guarantees that the FPGA can provide.

An additional challenge is the communication bridge between the two chips. Writing an I2C master in Verilog for this FPGA would consume a significant portion of the available LUTs and is difficult to verify without simulation infrastructure. Instead, a UART link was designed as the communication method, which is straightforward to implement in RTL and well-supported on the RP2040 side through the Arduino Serial2 interface.

The Vicharak Shrike Lite platform also introduces practical challenges such as volatile FPGA configuration (bitstream is lost on power cycle), pin mapping between FPGA GPIOs and the RP2040 header, and the need to manage the LittleFS filesystem upload workflow alongside sketch uploads during development.

4. Hardware and Software Tools Used

4.1 Hardware

Component	Description	Role in Project
Vicharak Shrike Lite	SLG47910V FPGA + RP2040 on single board	Main platform
Renesas SLG47910V	1120 LUT FPGA, 50 MHz internal oscillator	RTL execution
Raspberry Pi RP2040	Dual-core ARM Cortex-M0+ at 200 MHz	Firmware and UI
SH1106 OLED (128x64)	I2C OLED display module	Timer display
Passive Buzzer	5V passive piezo buzzer	Audio feedback
Vibration Motor Module	3-pin module with onboard MOSFET driver	Haptic feedback
Green and Red LEDs	Standard 3mm or 5mm LEDs with resistors	State indicators
Perfboard (9x15 cm)	Double-sided perforated PCB	Prototype assembly

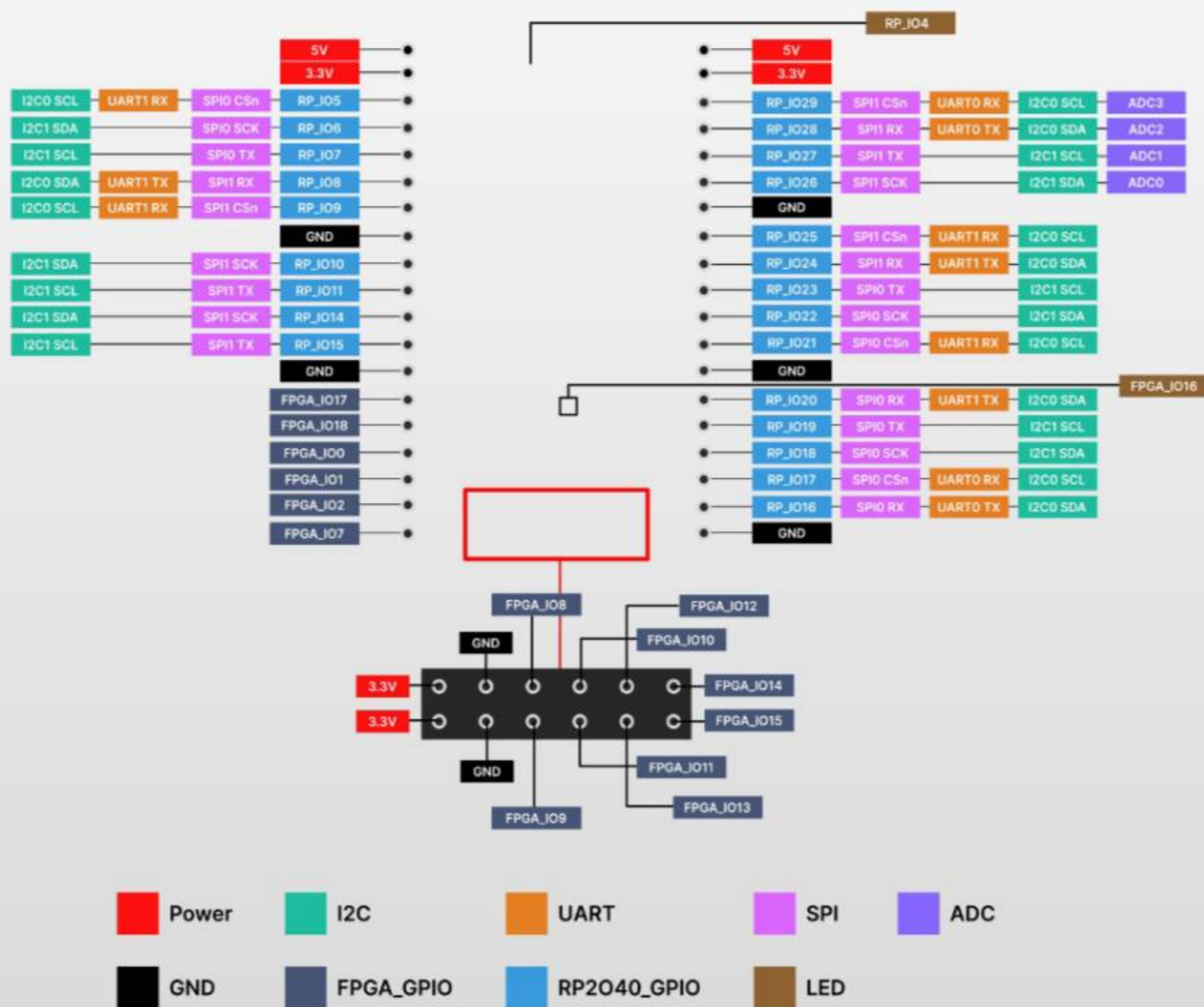
4.2 Software and Tools

Tool	Purpose
Renesas Go Configure Software Hub	Verilog synthesis, place and route, bitstream generation for SLG47910V
Arduino IDE 2	RP2040 firmware development using Arduino framework
Intel Quartus Prime 23.1	Compilation, RTL View and STA
Shrike.h Library	Board support package for Shrike Lite in Arduino IDE FPGA bitstream flashing from RP2040 over SPI
Adafruit SH110X Library	OLED display driver for SH1106 controller
Adafruit GFX Library	Graphics primitives for OLED rendering
arduino-littlefs-upload Plugin	LittleFS file system upload tool for Arduino IDE 2
LittleFS	Filesystem on RP2040 flash for storing FPGA bitstream



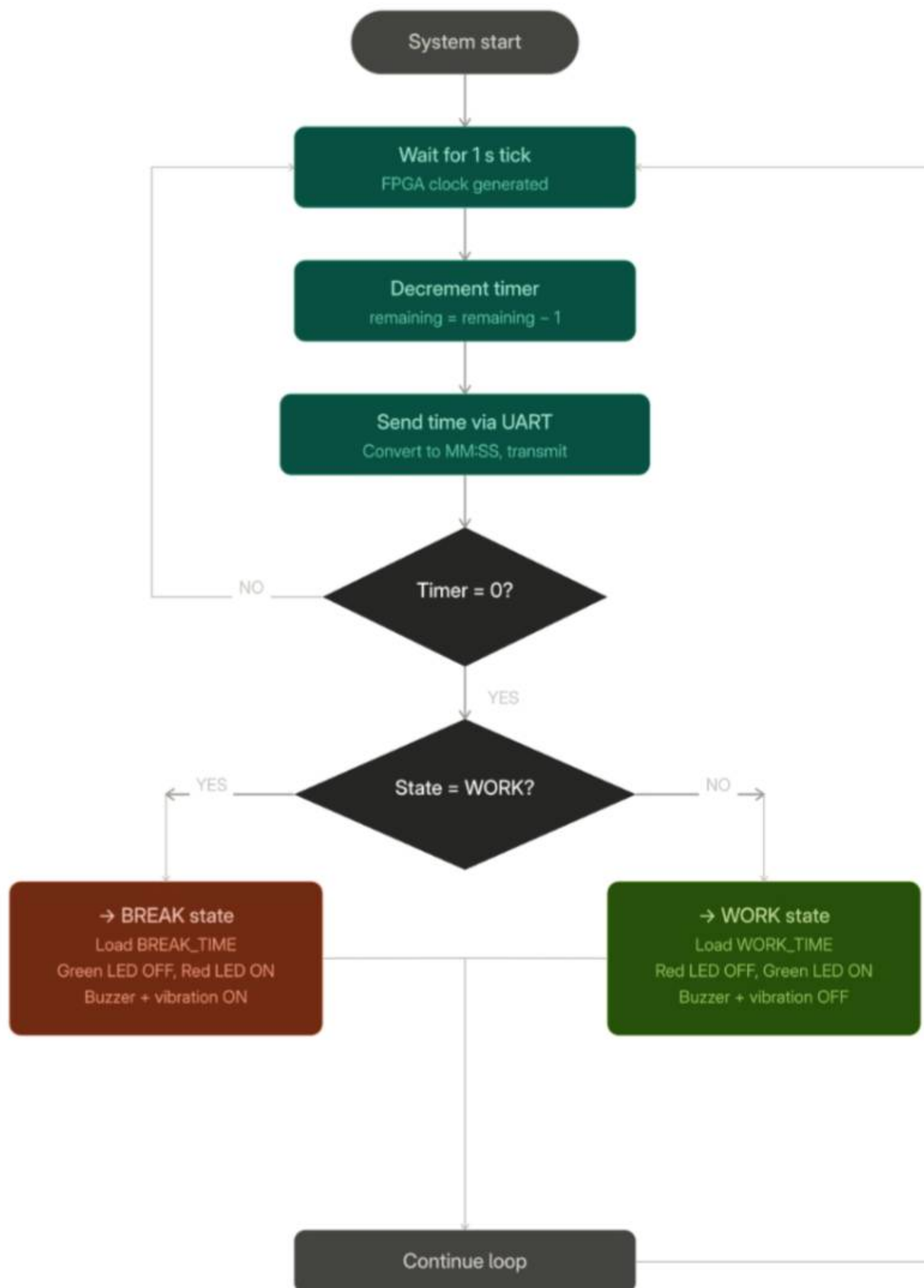
Pinout Diagram

SHRIKE - LITE

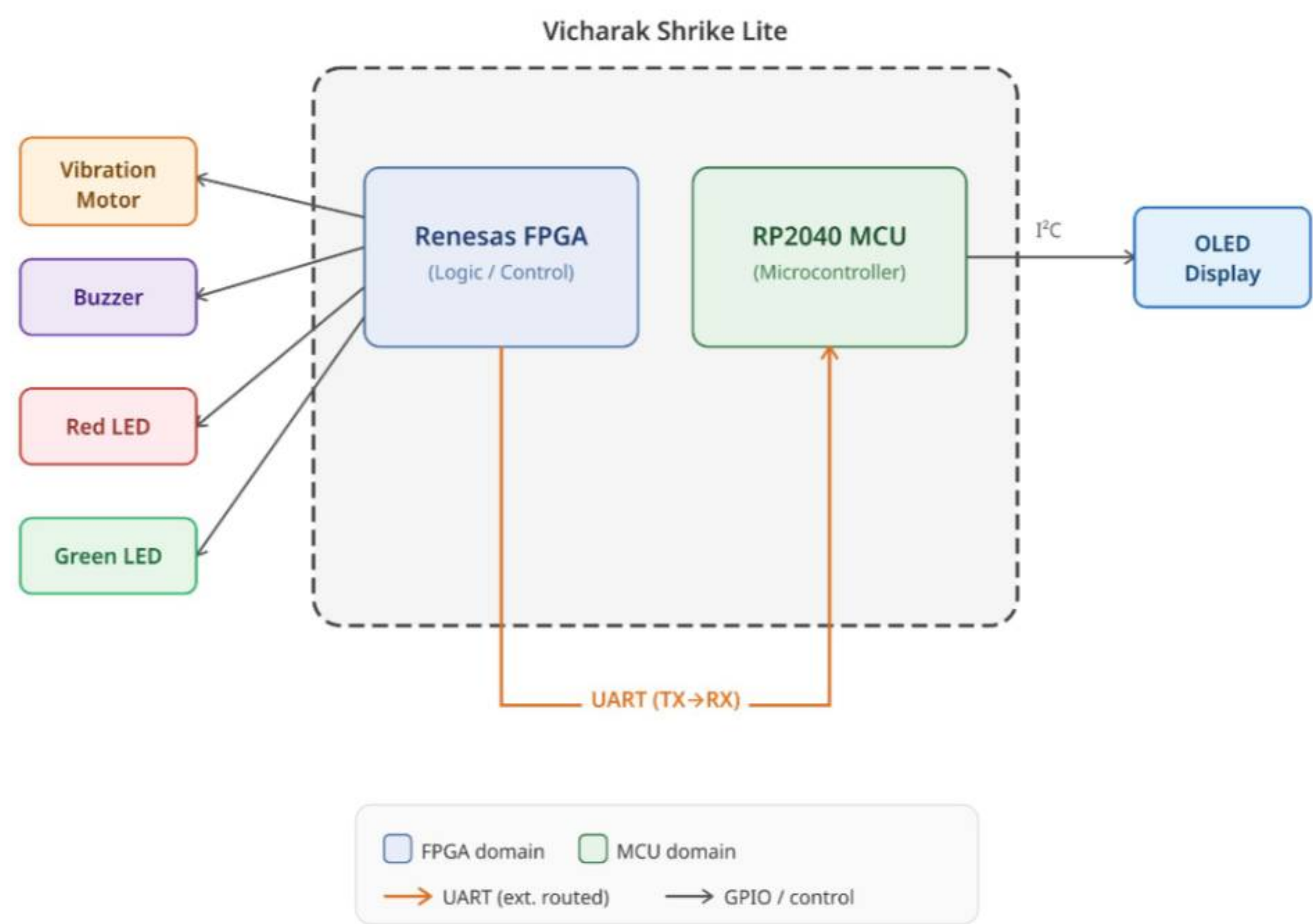


NOTE: "All GPIO's are 3.3V compatible"

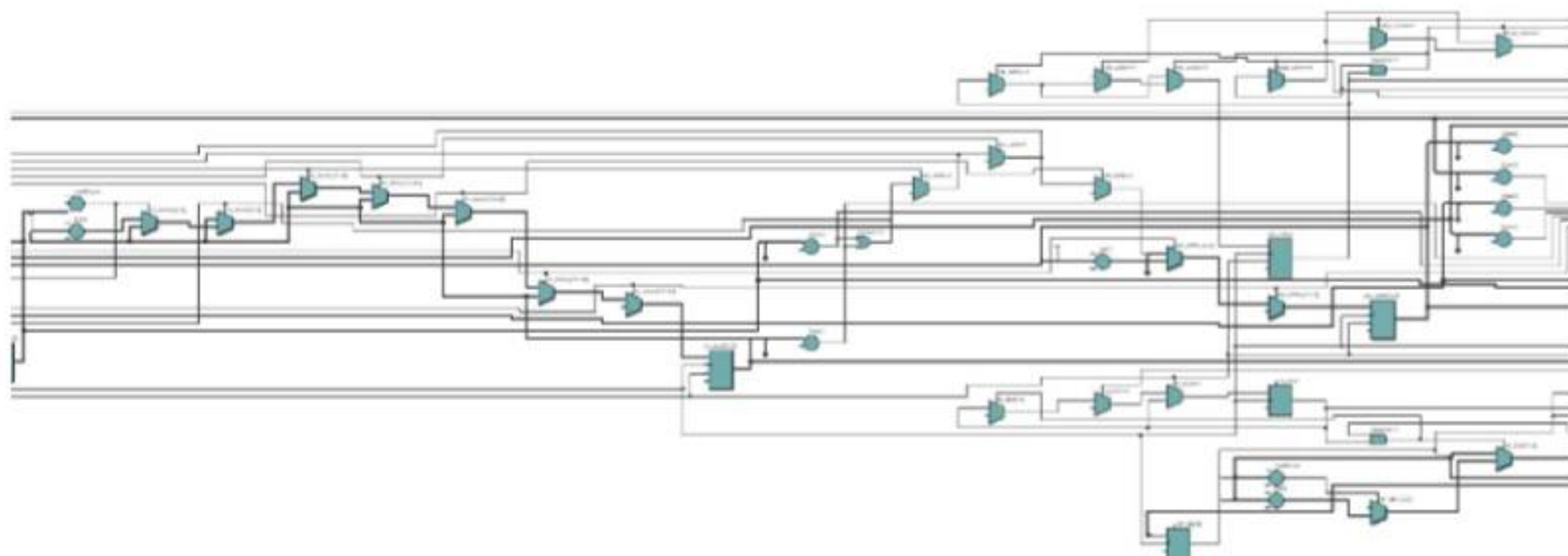
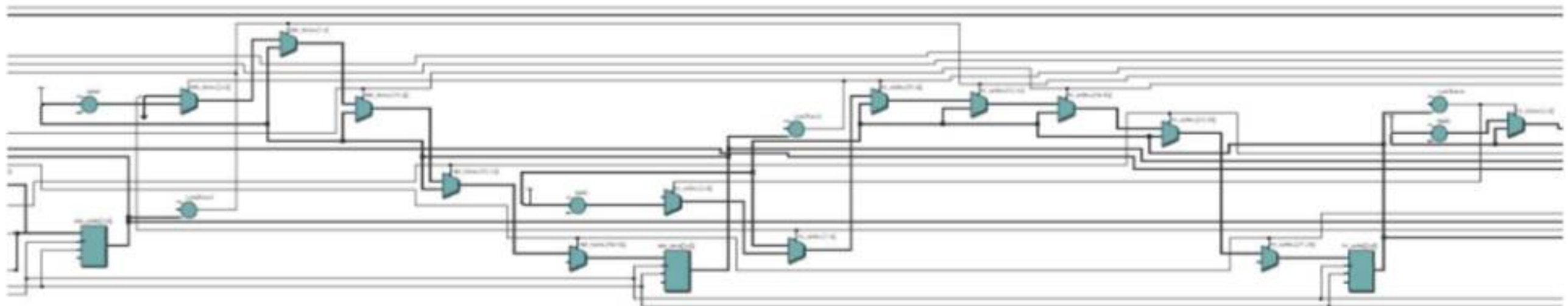
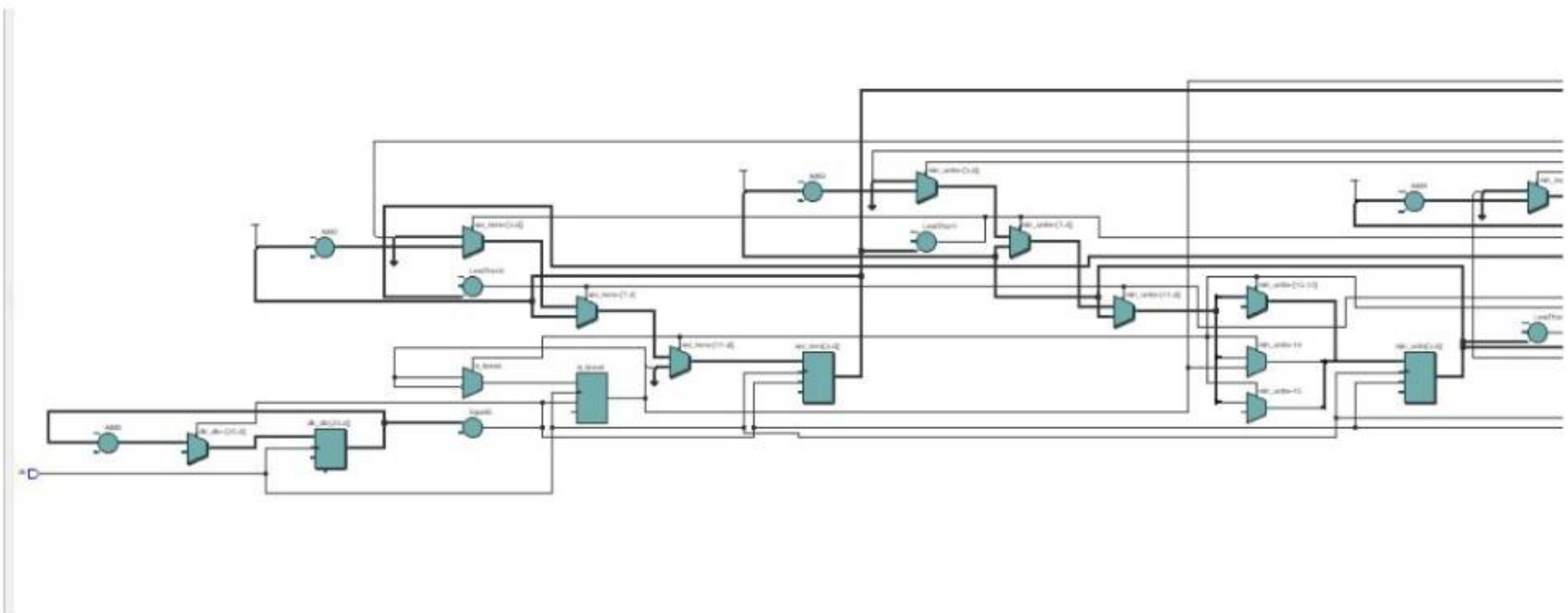
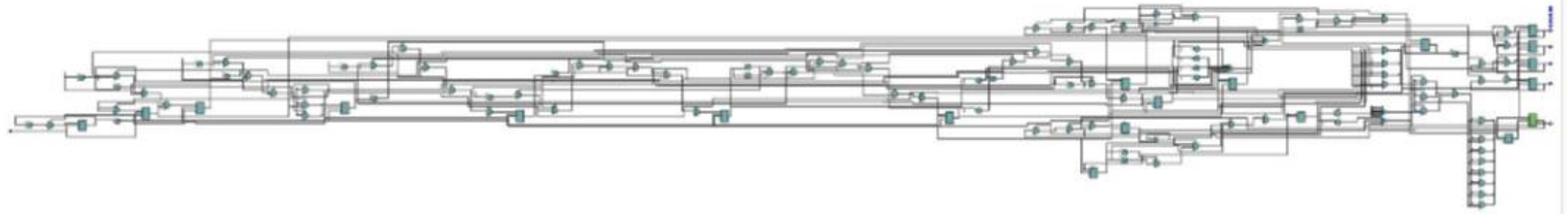
5. Design Methodology — Flowchart

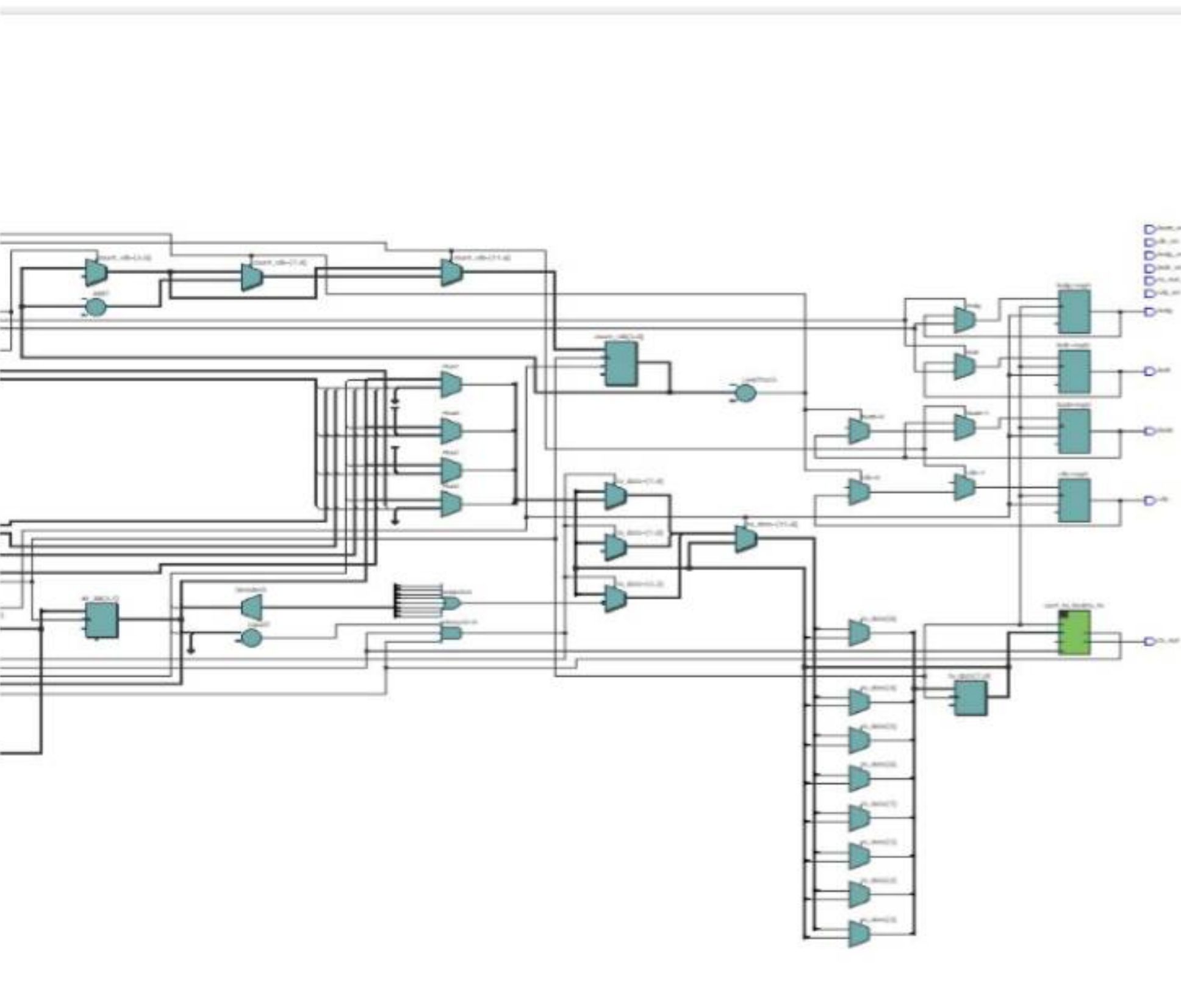


6. Block Diagram



7. Proposed Design - RTL View





8. HDL Code

Portlist Declarations

```
`timescale 1ns / 1ps

(* top *) module Pomodoro_FPGA (
    (* iopad_external_pin, clkbuf_inhibit *) input clk,
    (* iopad_external_pin *) output tx_out,
    (* iopad_external_pin *) output tx_out_en,
    (* iopad_external_pin *) output clk_en,
    (*iopad_external_pin *) output reg ledg=1'b0,
    (*iopad_external_pin *) output ledg_en,
    (*iopad_external_pin *) output reg ledr=1'b0,
    (*iopad_external_pin *) output ledr_en,
    (*iopad_external_pin *) output reg buzz=1'b0,
    (*iopad_external_pin *) output buzz_en,
    (*iopad_external_pin *) output reg vib=1'b0,
    (*iopad_external_pin *) output vib_en
);

    assign clk_en      = 1'b1;
    assign tx_out_en   = 1'b1;
    assign ledg_en     = 1'b1;
    assign ledr_en     = 1'b1;
    assign buzz_en     = 1'b1;
    assign vib_en      = 1'b1;
    wire n_rst=1'b1;
```


Clock Divider

```
26 // 1 Hz clk divider
27 reg [25:0] clk_div;
28 wire tick_1hz = (clk_div == 26'd49999999);
29
30 always @(posedge clk or negedge n_rst) begin
31     if (!n_rst)
32         clk_div <= 26'd0;
33     else if (tick_1hz)
34         clk_div <= 26'd0;
35     else
36         clk_div <= clk_div + 1;
37 end
38
39
40 wire active = 1'b1;
41
```

Timer Module – State machine with work and break states

```
44 // Timer
45 reg [3:0] sec_units=4'd0;
46 reg [3:0] sec_tens=4'd0;
47 reg [3:0] min_units=4'd1;
48 reg [3:0] min_tens=4'd0;
49 reg [3:0] hr_units=4'd0;
50 reg [3:0] hr_tens=4'd0;
51
52 reg [3:0] count_vib=4'd0;
53 reg vib_active = 1'b0;
54 reg buzz_active = 1'b0;
55 reg is_break = 1'b0;
56
```



```

57 always @(posedge clk) begin
58     if (active && tick_1hz) begin
59         if (hr_tens==0 && hr_units==0 && min_tens==0 && min_units==0 && sec_tens==0 && sec_units==0) begin
60             if (!is_break) begin
61
62                 is_break<=1'b1;
63                 hr_tens <= 0; hr_units <= 0;
64                 min_tens <= 0; min_units <= 2;
65                 sec_tens <= 0; sec_units <= 0;
66                 ledr <= 1'b1;
67                 ledg <= 1'b0;
68                 is_break <= 1'b1;
69             end else begin
70                 ledr <= 1'b0;
71                 ledg <= 1'b1;
72                 is_break<=1'b0;
73                 min_tens <= 0;
74                 min_units <= 0;
75                 sec_tens <= 3;
76                 sec_units <= 0;
77                 hr_tens <= 0; hr_units <= 0;
78             end
79         end else begin
80             // Countdown
81             if (sec_units > 0) sec_units <= sec_units - 1;
82             else begin
83                 sec_units <= 9;
84                 if (sec_tens > 0) sec_tens <= sec_tens - 1;
85                 else begin
86                     sec_tens <= 5;
87                     if (min_units > 0) min_units <= min_units - 1;
88                     else begin
89                         min_units <= 9;
90                         if (min_tens > 0) min_tens <= min_tens - 1;
91                         else begin
92                             if (hr_tens != 0 || hr_units != 0) begin
93                                 min_tens <= 5;
94                                 if (hr_units > 0) hr_units <= hr_units - 1;
95                                 else begin hr_units <= 9; hr_tens <= hr_tens - 1; end
96                             end else begin
97                                 min_tens <= 0; min_units <= 0;
98                                 sec_tens <= 0; sec_units <= 0;
99                             end
100                         end
101                     end
102                 end
103             end
104         end
105     end
106 end
107

```


Vibration motor & Buzzer trigger logic

```
108 //Vibration and buzzer
109 always @(posedge clk) begin
110     if (tick_1hz) begin
111         //Check the timer state to trigger vibration and buzzer
112         if (hr_tens==0 && hr_units==0 && min_tens==0 && min_units==0 &&
113             sec_tens==0 && sec_units==0 && is_break && !vib_active&&!buzz_active) begin
114             vib_active <= 1'b1;
115             buzz_active <= 1'b1;
116             count_vib <= 4'd0;
117         end
118
119         if (vib_active&&buzz_active) begin
120             if (count_vib < 10) begin
121                 vib <= ~vib;
122                 buzz<=~buzz;
123                 count_vib <= count_vib + 1;
124             end else begin
125                 vib <= 1'b0;
126                 buzz<=1'b0;
127                 vib_active <= 1'b0;
128                 buzz_active<=1'b0;
129             end
130         end else begin
131             vib <= 1'b0;
132         end
133     end
134 end
135 end
```

Uart Transmitter Module

```
137 //UART Transmitter (9600 Baud)
138 reg [7:0] tx_data;
139 reg tx_start;
140 wire tx_busy;
141
142 // String sending state machine: "HH:MM:SS\n"
143 reg [3:0] str_idx;
144
```



```

145 always @(posedge clk or negedge n_rst) begin
146     if (!n_rst) begin
147         str_idx <= 4'd1;
148         tx_start <= 1'b1;
149     end else begin
150         if (tick_1hz && active) begin
151             str_idx <= 4'd1;
152         end else if (str_idx != 0 && !tx_busy && !tx_start) begin
153             tx_start <= 1'b1;
154             case (str_idx)
155                 4'd1: tx_data <= {4'h3, hr_tens};
156                 4'd2: tx_data <= {4'h3, hr_units};
157                 4'd3: tx_data <= 8'h3A;
158                 4'd4: tx_data <= {4'h3, min_tens};
159                 4'd5: tx_data <= {4'h3, min_units};
160                 4'd6: tx_data <= 8'h3A;
161                 4'd7: tx_data <= {4'h3, sec_tens};
162                 4'd8: tx_data <= {4'h3, sec_units};
163                 4'd9: tx_data <= 8'h0A;
164                 default: tx_data <= 8'h0A;
165             endcase
166         end else if (tx_start && tx_busy) begin
167             tx_start <= 1'b0;
168             if (str_idx < 9)
169                 str_idx <= str_idx + 1;
170             else
171                 str_idx <= 4'd0;
172         end
173     end
174 end

```

```

177     uart_tx_basic u_tx (
178         .clk(clk),
179         .n_rst(n_rst),
180         .start(tx_start),
181         .data(tx_data),
182         .tx_out(tx_out),
183         .busy(tx_busy)
184     );
185
186 endmodule

```


Shift register logic for data shifting via UART

```
188 module uart_tx_basic (  
189     input clk,  
190     input n_rst,  
191     input start,  
192     input [7:0] data,  
193     output reg tx_out,  
194     output reg busy  
195 );  
196     parameter CLKS_PER_BIT = 5208; // for 9600 baud rate  
197  
198     reg [12:0] clk_cnt;  
199     reg [3:0] bit_cnt;  
200     reg [8:0] shift_reg;  
201  
202     always @(posedge clk or negedge n_rst) begin  
203         if (!n_rst) begin  
204             tx_out <= 1'b1;  
205             busy <= 1'b0;  
206             clk_cnt <= 0;  
207             bit_cnt <= 0;  
208         end else begin  
209             if (start && !busy) begin  
210                 busy <= 1'b1;  
211                 clk_cnt <= 0;  
212                 bit_cnt <= 0;  
213                 shift_reg <= {1'b1, data};  
214                 tx_out <= 1'b0; // start bit  
215             end else if (busy) begin  
216                 if (clk_cnt < CLKS_PER_BIT - 1) begin  
217                     clk_cnt <= clk_cnt + 1;  
218                 end else begin  
219                     clk_cnt <= 0;
```



```

220         if (bit_cnt < 9) begin
221             tx_out    <= shift_reg[0];
222             shift_reg <= {1'b1, shift_reg[8:1]};
223             bit_cnt   <= bit_cnt + 1;
224         end else begin
225             busy    <= 1'b0;
226             tx_out  <= 1'b1;
227         end
228     end
229 end
230 end
231 end
232 endmodule

```

Testbench for timer logic

```

1  module pomodoro_timer(input clk,input rst,output reg work,output reg brk,output reg [5:0] sec);
2      parameter w_time=5;parameter b_time=3;
3      reg mode;reg [5:0] limit;
4      parameter w=1'b0;parameter b=1'b1;
5      always @(posedge clk or posedge rst)
6      begin
7          if(rst)
8          begin
9              mode<=w;sec<=0;limit<=w_time;work<=1;brk<=0;
10             end
11             else
12             begin
13                 if(sec==limit-1)
14                 begin
15                     sec<=0;
16                     if(mode==w)
17                     begin
18                         mode<=b;limit<=b_time;work<=0;brk<=1;
19                     end
20                     else
21                     begin
22                         mode<=w;limit<=w_time;work<=1;brk<=0;
23                     end
24                 end
25                 else
26                 begin
27                     sec<=sec+1;
28                 end
29             end
30         end
31     endmodule
32 module tb_pomodoro_timer;
33     reg clk;reg rst;
34     wire work;wire brk;wire [5:0] sec;
35     pomodoro uut(.clk(clk),.rst(rst),.work(work),.brk(brk),.sec(sec));
36     initial begin clk=0;forever #5 clk=~clk;end
37     initial begin rst=1;#20;rst=0;#500;$stop;end
38 endmodule
39

```


9. MCU Code & Serial Monitor Output

```
1  #include <Wire.h>
2  #include <Shrike.h>
3  #include <Adafruit_GFX.h>
4  #include <Adafruit_SH110X.h>
5
6  Adafruit_SH1106G myoled(128, 64, &Wire, -1);
7  ShrikeFlash shrike;
8
9  void initDisplay() {
10     myoled.begin(0x3C);
11     myoled.clearDisplay();
12     myoled.fillScreen(SH110X_BLACK);
13     myoled.setTextColor(SH110X_WHITE);
14     myoled.setTextSize(1);
15     myoled.setCursor(16, 24);
16     delay(100);
17 }
18
19 void initFPGA() {
20     shrike.begin();
21     if (shrike.flash("/uartlogic.bin")) {
22         Serial.println("Bitstream loaded.");
23         myoled.clearDisplay();
24         myoled.fillScreen(SH110X_BLACK);
25         myoled.setTextColor(SH110X_WHITE);
26         myoled.setTextSize(2);
27
28         delay(100);
29     } else {
30         Serial.println("Flash failed!");
31     }
32 }
33
```



```
33
34 void setup() {
35     Serial.begin(115200);
36     delay(3000);
37     Serial.println("Started Debugging");
38
39     Wire.setSCL(29);
40     Wire.setSDA(28);
41     Wire.begin();
42
43     initDisplay();
44     initFPGA();
45
46     Serial2.setRX(9);      // FPGA IO7 → Pico IO9
47     Serial2.setTX(20);
48     Serial2.begin(9600);
49     Serial.println("UART ready");
50 }
51
```

```
52 void loop() {
53     static unsigned long prev_time = 0;
54     static unsigned int uptime = 0;
55
56     if (millis() - prev_time > 1000) {
57         prev_time = millis();
58         uptime++;
59         Serial.println("Uptime: " + String(uptime));
60         Serial.println("Data in buffer: " + String(Serial2.available()));
61     }

```



```

62
63  ✓   if (Serial2.available()) {
64       String data = Serial2.readStringUntil('\n');
65       data.trim();
66       Serial.println("Received: " + data);
67  ✓   if (data.length() >= 8) {
68       myoled.clearDisplay();
69       myoled.fillScreen(SH110X_BLACK);
70       myoled.setTextColor(SH110X_WHITE);
71       myoled.setTextSize(2);
72       myoled.setCursor(16, 24);
73       myoled.print(data);
74       myoled.display();
75       }
76   }
77 }

```

Output

LittleFS Upload

Serial Monitor X

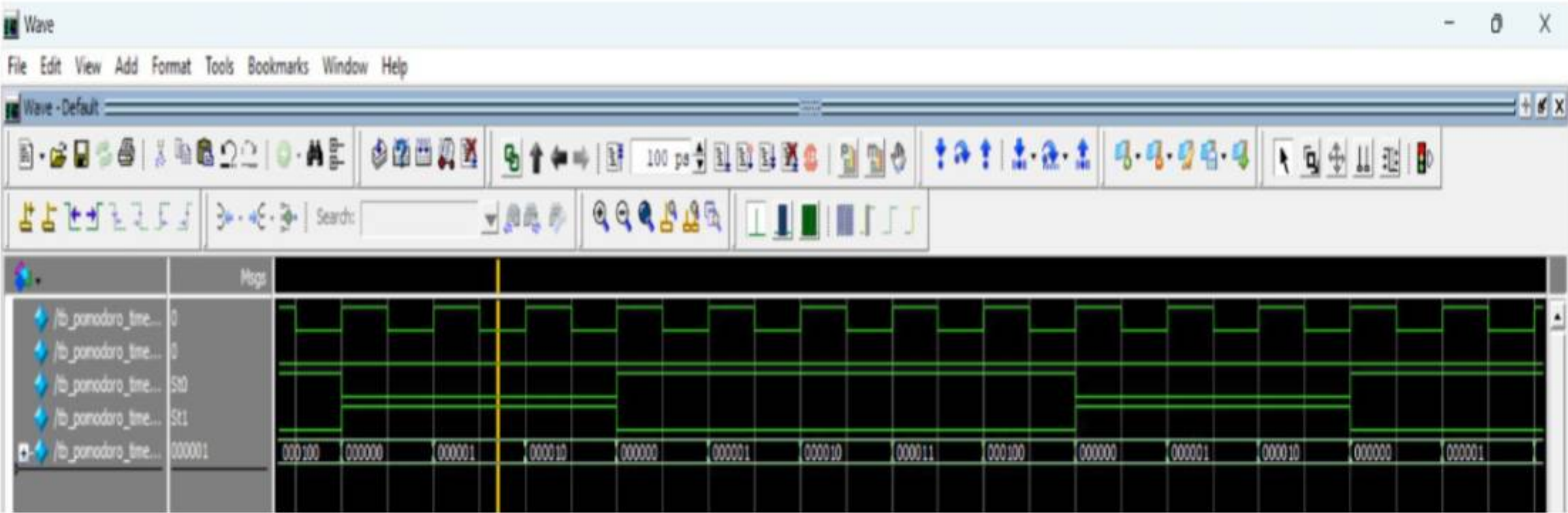
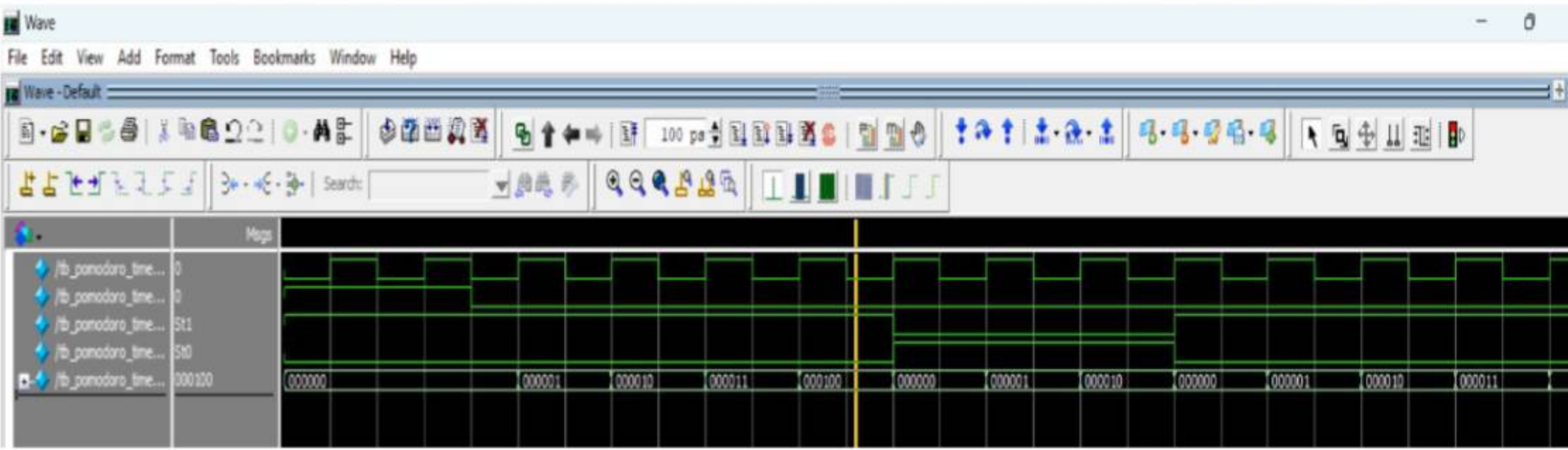
Message (Enter to send message to 'Vicharak Shrike-Lite' on 'COM7')

```

Uptime: 19
Data in buffer: 0
Received: 00:01:42
Uptime: 20
Data in buffer: 0
Received: 00:01:41
Uptime: 21
Data in buffer: 0
Received: 00:01:40
Uptime: 22
Data in buffer: 0
Received: 00:01:39
Uptime: 23
Data in buffer: 0
Received: 00:01:38
Uptime: 24
Data in buffer: 0

```


10. Simulation Waveform



11. Static Timing Analysis Report (STA Report)

Performed STA for the Altera DE2-115 FPGA Board

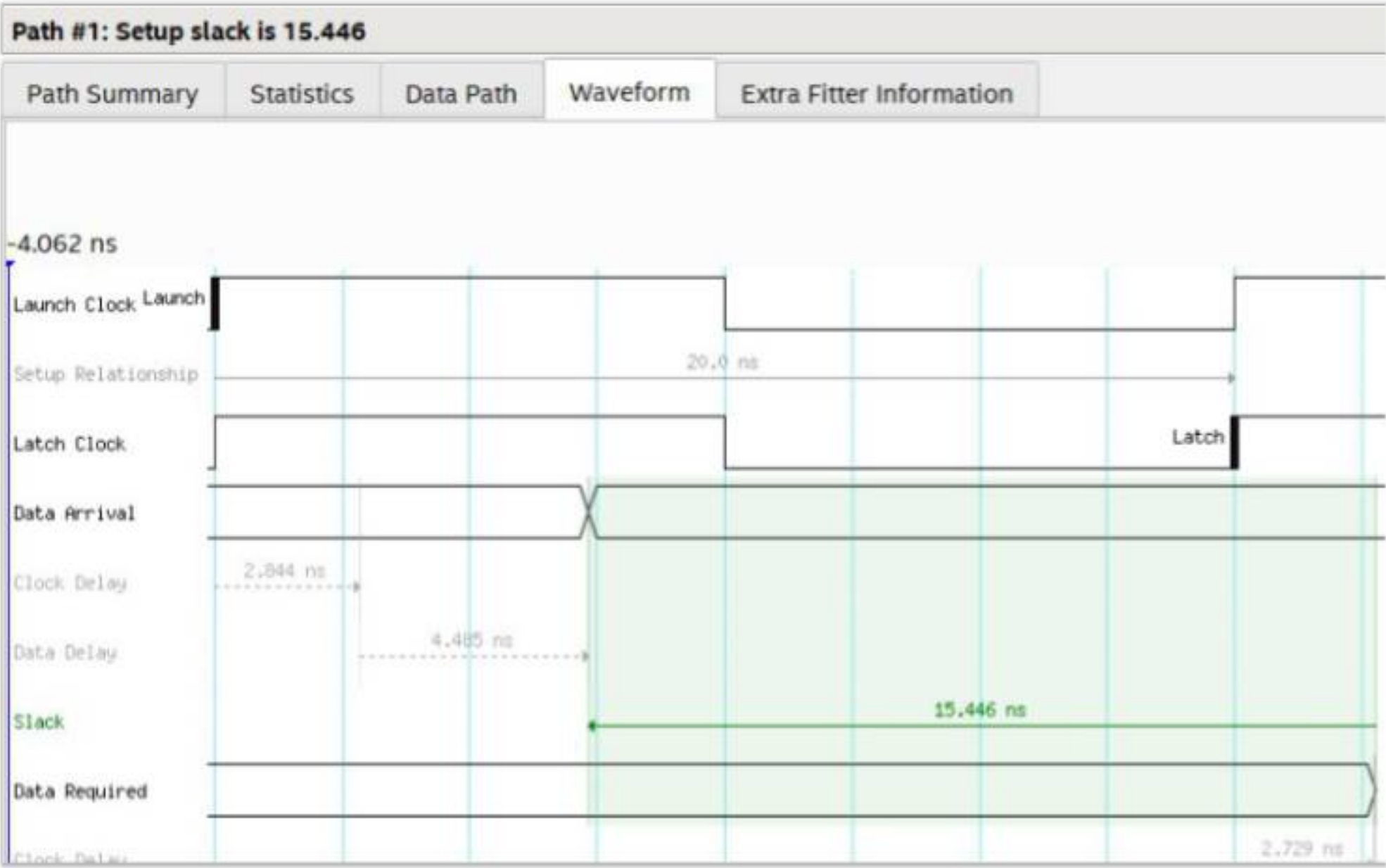
⚠ Number of processors has not been specified which may cause overloading on shar
① Parallel compilation is enabled and will use 6 of the 6 processors detected
① Low junction temperature is 0 degrees C
① High junction temperature is 85 degrees C
tcl> create_clock -name shrike_clk -period 20.000 [get_ports {clk}]
tcl> update_timing_netlist
⚠ The following clock transfers have no clock uncertainty assignment. For more ac
tcl>

SETUP REPORT

	Clock	Slack	End Point TNS
1	shr...clk	15.446	0.000

Slow 1200mV								
Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	15.446	clk_div[10]	sec_...s[3]	shrike_clk	shrike_clk	20.000	-0.087	4.485

Path #1: Setup slack is 15.446			
Path Summary		Statistics	Waveform
	Property	Value	
1	From Node	clk_div[10]	
2	To Node	sec_tens[3]	
3	Launch Clock	shrike_clk	
4	Latch Clock	shrike_clk	
5	Data Arrival Time	7.329	
6	Data Required Time	22.775	
7	Slack	15.446	



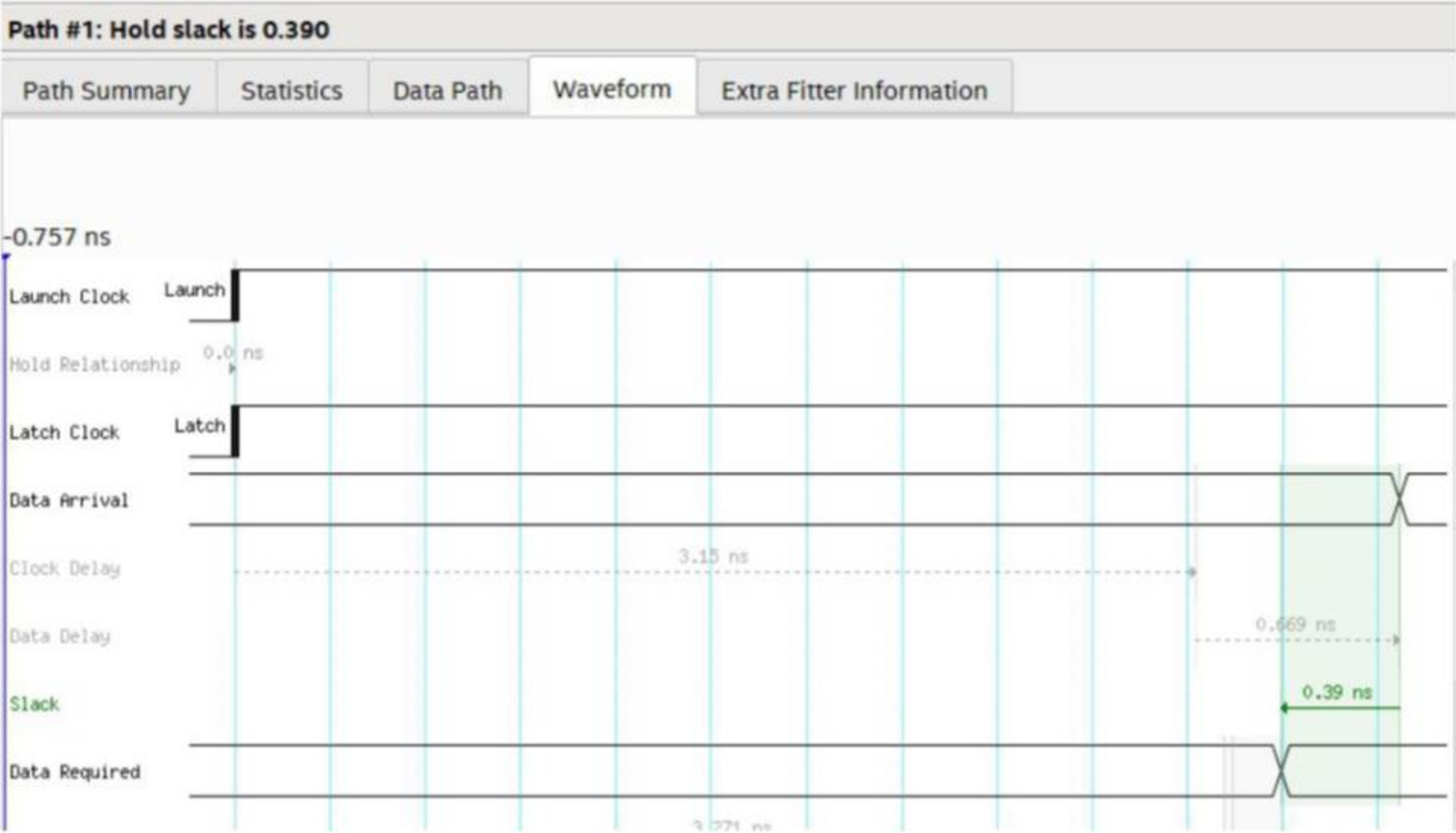
Setup Slack = 15.446 ns

HOLD REPORT

	Clock	Slack	End Point TNS
1	shr...clk	0.390	0.000

Path #1: Hold slack is 0.390					
Path Summary		Statistics	Data Path	Waveform	Ex
	Property	Value			
1	From Node	uart_tx_basic:u_tx busy			
2	To Node	uart_tx_basic:u_tx busy			
3	Launch Clock	shrike_clk			
4	Latch Clock	shrike_clk			
5	Data Arrival Time	3.819			
6	Data Required Time	3.429			
7	Slack	0.390			

Command Info		Summary of Paths							
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay	
1	0.390	uart_t...x busy	uart...busy	shrike_clk	shrike_clk	0.000	0.093	0.669	



Hold Slack = 0.390 ns

Fmax Summary Report

	Fmax	Restricted Fmax	Clock Name	Note
1	219.59 MHz	219.59 MHz	shrike_clk	

Clock to output times (tc2q)

	Data Port	Clock Port	Rise	Fall	Clock Edge	Clock Reference
1	buzz	shrike_clk	6.762	6.760	Rise	shrike_clk
2	ledg	shrike_clk	8.612	8.670	Rise	shrike_clk
3	ledr	shrike_clk	7.325	7.281	Rise	shrike_clk
4	tx_out	shrike_clk	6.594	6.560	Rise	shrike_clk
5	vib	shrike_clk	6.742	6.738	Rise	shrike_clk

12. FPGA Implementation Screenshots and Demo

The system was assembled on a 9x15 cm double-sided perfboard with the FPGA module, OLED display, buzzer, vibration motor, and LED indicators.

The FPGA bitstream was successfully synthesized and loaded onto the SLG47910V via the RP2040 using the Shrike library at every boot. The OLED display correctly shows the timer value received over UART, updating once per second. The green and red LEDs respond to the work and break state transitions respectively. The buzzer and vibration motor activate at the end of each session as intended.

I/O Planner

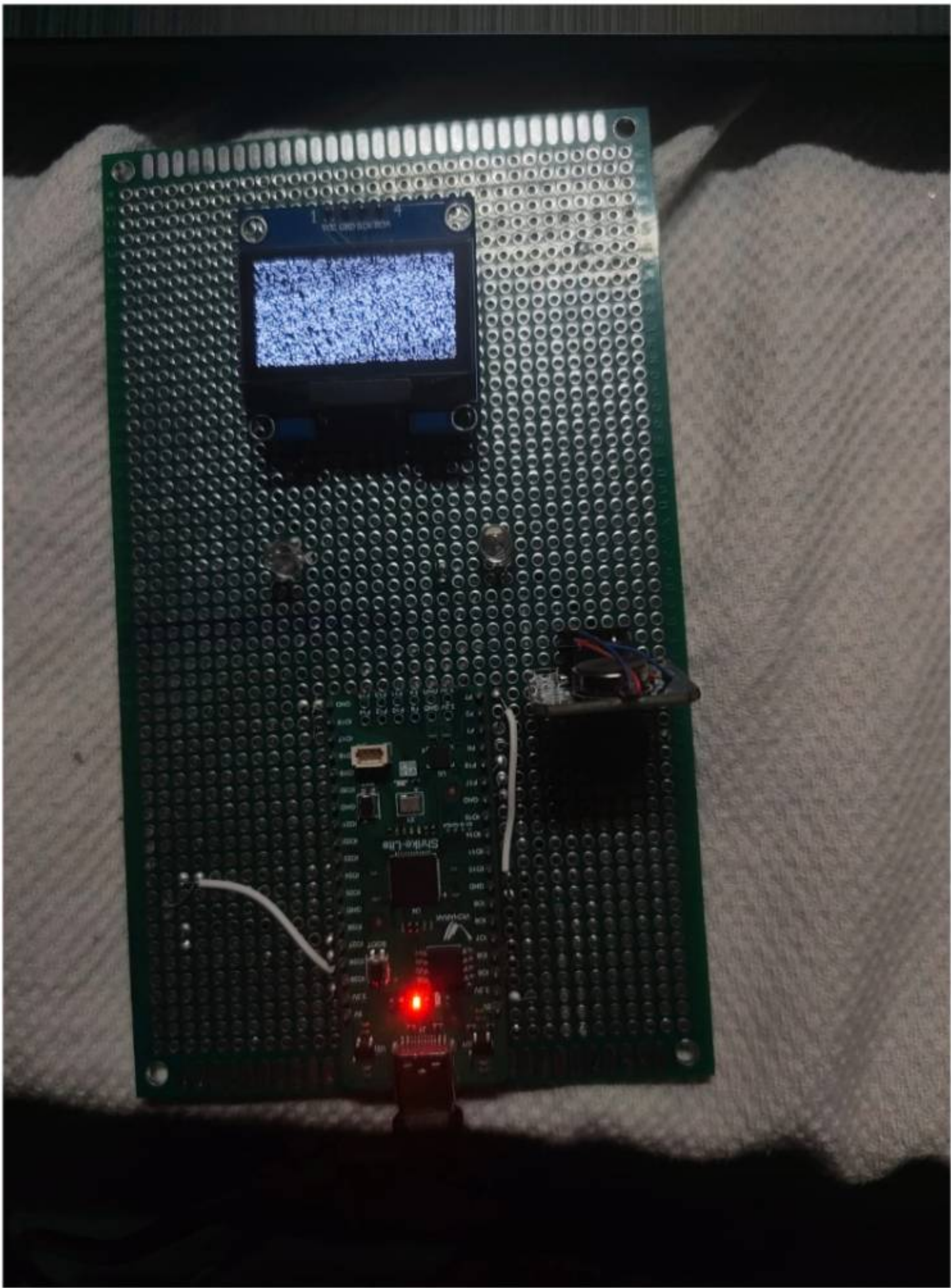
OSC_EN	Output	clk_en
OSC_CLK	Input	clk
LOGIC_AS_CLK1	Input	
LOGIC_AS_CLK0	Input	
GPI018_OUT [PIN 9]	Output	ledr
GPI018_OE [PIN 9]	Output	ledr_en
GPI018_IN [PIN 9]	Input	
GPI017_OUT [PIN 8]	Output	ledg
GPI017_OE [PIN 8]	Output	ledg_en
GPI08_OE [PIN 23]	Output	
GPI08_IN [PIN 23]	Input	
GPI07_OUT [PIN 20]	Output	tx_out
GPI07_OE [PIN 20]	Output	tx_out_en
GPI07_IN [PIN 20]	Input	
GPI06_OUT [PIN 19]	Output	
GPI06_OE [PIN 19]	Output	
GPI06_IN [PIN 19]	Input	
GPI05_OUT [PIN 18]	Output	

GPI02_IN [PIN 15]	Input	
GPI01_OUT [PIN 14]	Output	vib
GPI01_OE [PIN 14]	Output	vib_en
GPI01_IN [PIN 14]	Input	
GPI00_OUT [PIN 13]	Output	buzz
GPI00_OE [PIN 13]	Output	buzz_en
GPI00_IN [PIN 13]	Input	

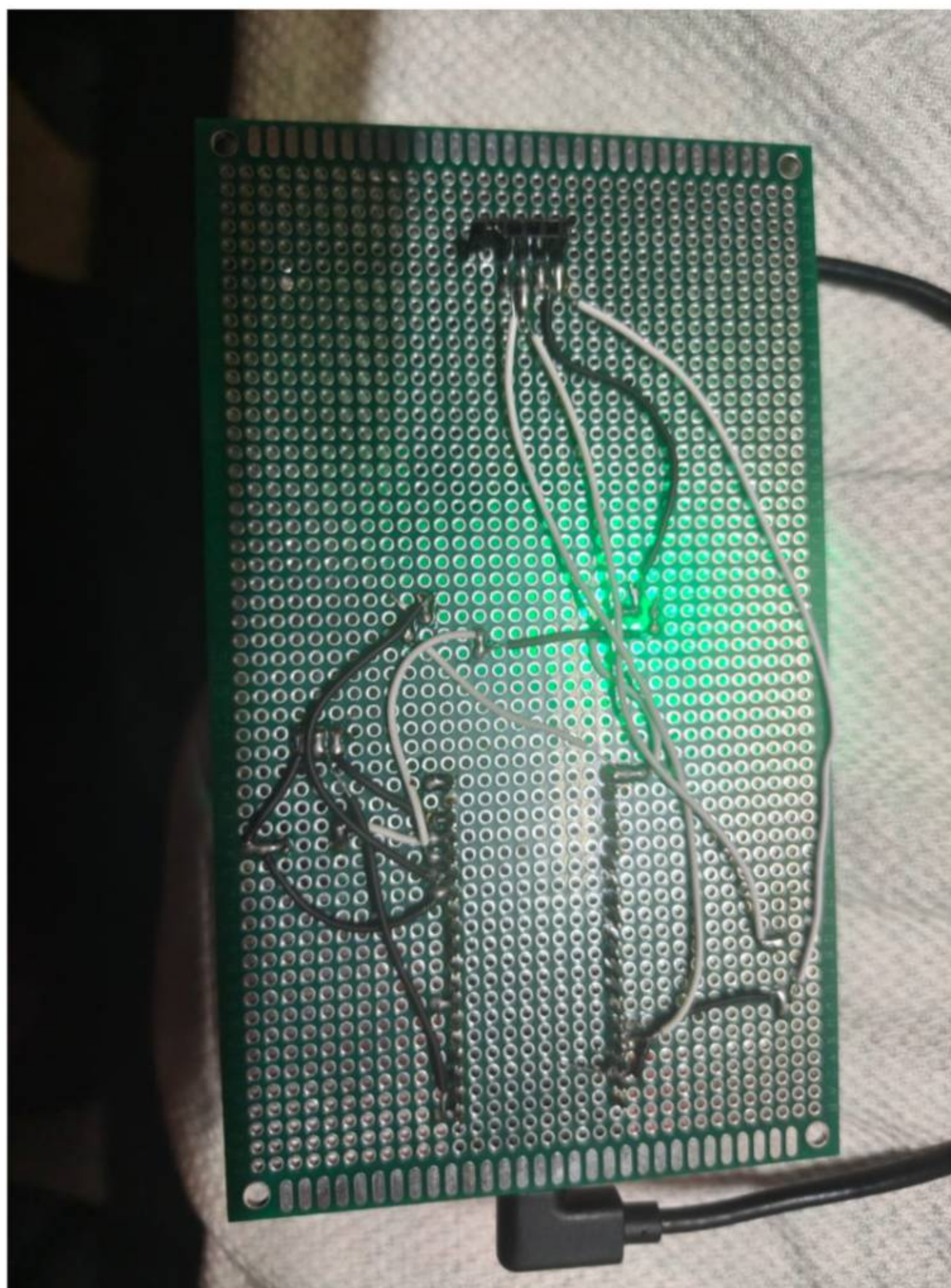
The demo videos of the working prototype are available in the following link:

https://drive.google.com/drive/folders/1ojC_WO2B6vwgpKh14xuvw-Em7hV2OEVN?usp=sharing

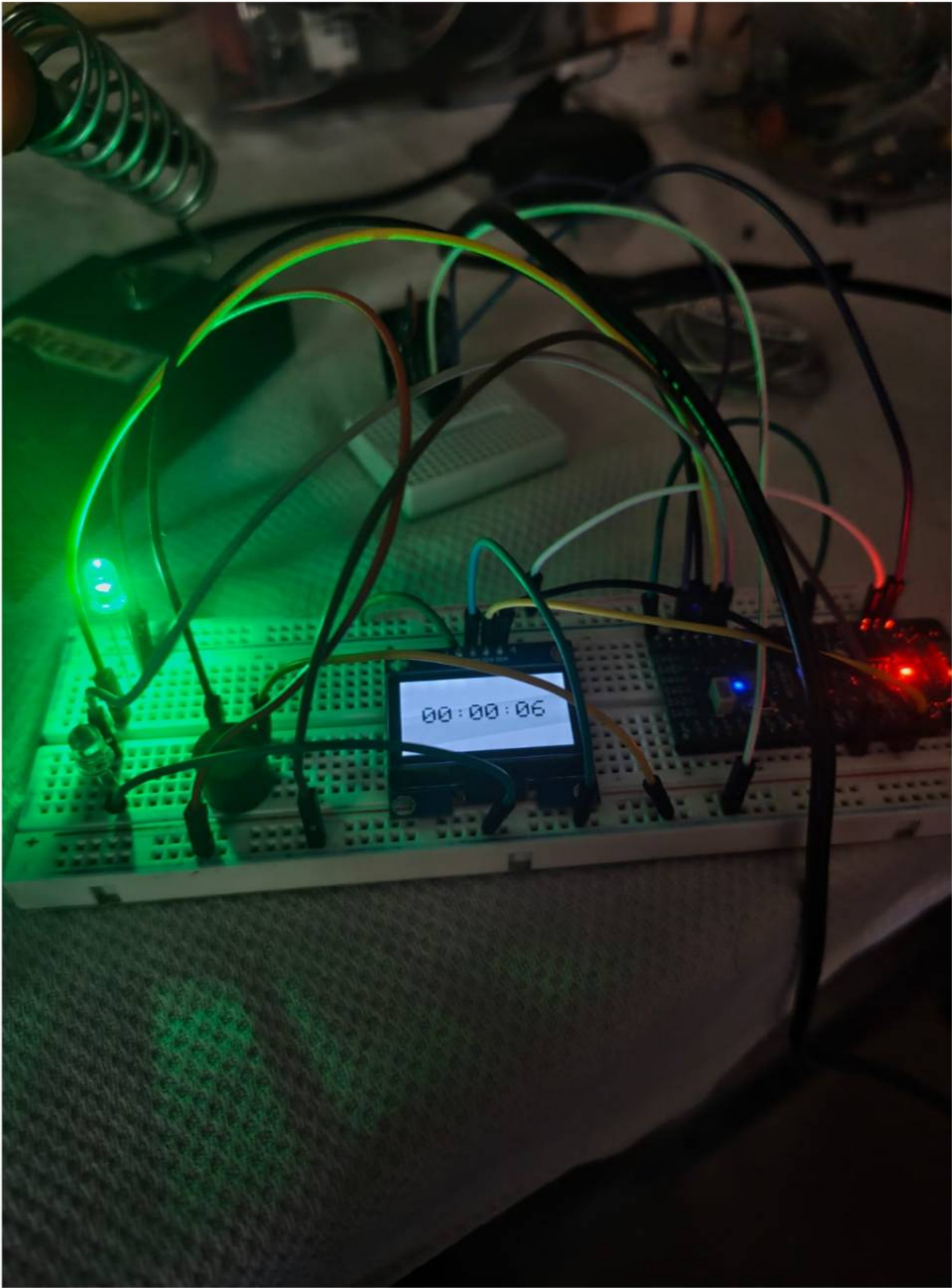
FPGA in Boot Mode Configuration (Before bitstream has been loaded)



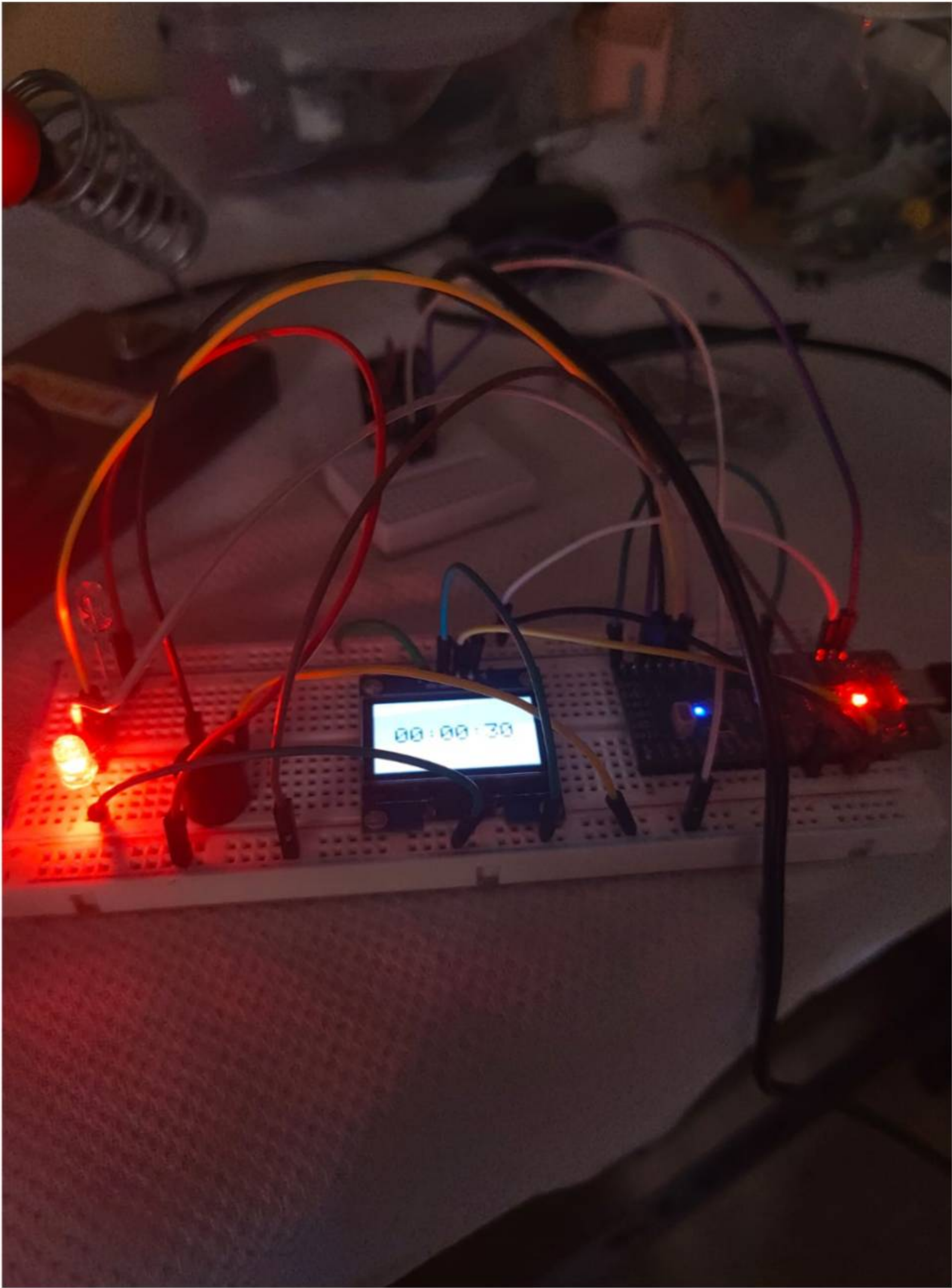




Work Mode (2 minutes)



Break Mode (30 seconds)



13. Future Improvements and Scope

13.1 Hardware Enhancements

1. The project can be migrated from a perfboard to a custom PCB to improve reliability, reduce wire length, and give the device a more professional appearance. A two-layer PCB with a dedicated power plane would significantly reduce noise.
2. A WS2812B RGB LED strip can be added and driven using a custom Verilog WS2812 driver. This would require implementing a state machine that generates the precise 800 kHz single-wire signal with the correct T0H, T0L, T1H, and T1L timings. Each LED in the strip can represent one minute of the work session, filling up as time progresses.
3. A rotary encoder can be connected to the RP2040 to allow the user to configure the work duration before starting the timer. The RP2040 would read the encoder, display a configuration screen on the OLED, and send the selected duration to the FPGA over a reverse UART channel before the timer starts.
4. A Li-Po battery with a TP4056 USB-C charging module and an MT3608 boost converter to step 3.7V up to 5V would make the device fully portable and self-contained.

13.2 RTL Enhancements

1. The current design uses a single active work state followed by a break. A full Pomodoro FSM can be implemented in Verilog with distinct states for **IDLE**, **WORK**, **SHORT BREAK**, **LONG BREAK**, and **ALARM**. After four work sessions the FSM would automatically transition to a longer break.
2. A UART receiver module can be added to the FPGA to accept configuration packets from the RP2040. This would allow bidirectional communication where the RP2040 sends a start command and duration value and the FPGA loads them into its countdown registers.
3. PWM tone generation for the buzzer can be implemented in RTL to produce distinct tones for different events.

13.3 Software Enhancements

1. The OLED display layout can be improved with a graphical progress bar that represents the remaining time as a shrinking bar across the bottom of the screen, rendered using the Adafruit GFX `drawFastHLine` function.
2. Session statistics can be tracked on the RP2040, such as the total number of completed sessions, and stored in its EEPROM. These can be displayed on a secondary screen accessible by pressing a button.

14. Individual Contribution from Team Members

Rohan Venkataraman – Hardware prototyping and configuration of UART transmission state machine between FPGA and MCU

Joseph Justin – Timer logic state machine & block diagram

Varun Shankar & Sanat Vashist – STA, testbench, simulation, flowchart

15. Social and Environmental Impact with Respect to SDGs

15.1 SDG 3 - Good Health and Well-Being

The Pomodoro technique is a productivity method that helps individuals manage cognitive fatigue and avoid burnout. By building a dedicated physical device for this purpose, the project promotes healthier work habits, particularly relevant for students and professionals who spend extended hours in front of screens. A physical timer has been shown to be more effective than software timers in reducing digital distractions, which directly contributes to mental well-being and stress reduction.

15.2 SDG 4 - Quality Education

The project itself is an educational exercise that bridges the gap between theoretical HDL concepts taught in coursework and practical hardware implementation. The Vicharak Shrike Lite is an affordable platform specifically designed to make FPGA technology accessible to students in developing countries. Demonstrating a working co-design project using this board contributes to the broader goal of democratizing access to hardware design education. The open-source nature of the Shrike ecosystem encourages peer learning and knowledge sharing.

15.3 SDG 12 - Responsible Consumption and Production

The project was implemented on a low-cost development board priced under 370 INR, minimising material consumption. The Li-Po battery with a USB-C charging module allows the device to be recharged hundreds of times rather than relying on disposable batteries, reducing electronic waste. The compact form factor also means minimal material usage in the final assembly.

15.4 SDG 17 - Partnerships for the Goals

This project makes use of entirely open-source tools and libraries including the Arduino framework, Adafruit GFX, the Shrike library, and the LittleFS upload plugin. The Vicharak Shrike Lite board itself is designed and manufactured in India with open hardware documentation. Contributing project code and documentation back to the community supports the collaborative ecosystem that enables future students and makers to build on this work.